Soutenance Projet 6 OpenClassrooms

Fishoye

Créez un site accessible pour une plateforme de photographes

Présenter par Ghani

PROJET 6

Fishoye

01

Présentation

02

Pages

03

Les factories

Présentation

Avec votre cheffe de projet Amanda et le Designer UI, vous venez de faire une réunion de lancement du projet avec un nouveau client, FishEye. FishEye est un site web qui permet aux photographes indépendants de présenter leurs meilleurs travaux. Ils ont récemment levé des fonds et aimeraient mettre à jour leur site web.

Après la réunion de lancement, vous voyez un mail de votre cheffe de projet concernant le projet FishEye. Vous l'ouvrez immédiatement :

Salut,

Merci d'avoir participé à la réunion de ce matin ! Pour résumer, notre objectif est de construire un prototype fonctionnel d'un nouveau site web que nous pourrons présenter à FishEye lors de notre prochaine réunion avec les clients. Tu seras chargé de fournir tout le HTML, le CSS et le JavaScript nécessaires au prototype. Notre équipe de back-end intégrera le système existant de FishEye une fois que tu auras terminé le code pour ta partie du projet.

Pour t’aider à démarrer, voici toutes les informations que j'ai recueillies auprès du client :

- Mes notes de réunion détaillant les principales fonctionnalités et les exigences techniques à mettre en œuvre.
- Les maquettes approuvées, développées par notre designer. Tu vas créer une page principale ainsi qu’une page avec les informations pour chaque photographe de l’échantillon.
- Des exemples de photos et de vidéos de FishEye, que tu devras utiliser pour la conception des pages.
- Des exemples de données au format JSON, que tu pourras utiliser pour créer les différentes pages des photographes de façon dynamique. Ce format imite la structure des données dans la base de données, donc ne modifie pas les données existantes.

J'avais demandé à Zoé de s’occuper de ce projet, mais après avoir rédigé un premier jet très rapidement, elle a dû passer à autre chose. Tu trouveras la base de son code sur ce repo avec une partie du HTML et CSS mis en place, ce qui devrait t’aider. Tu peux forker son repo pour te lancer dans le code.

Bien que le site web soit un prototype, il devrait correspondre aux maquettes et fonctionner correctement (pas d'erreur dans la console). Comme l'a mentionné le client lors de la réunion de lancement, sa priorité absolue est l'accessibilité. Veille à construire le site conformément aux exigences d'accessibilité indiquées dans mes notes.

Une fois que tu m'auras envoyé le repo GitHub avec le code complété, nous passerons en revue et vérifierons toutes les fonctionnalités des pages. Je te demanderai également de faire une démonstration de la navigation du site au moyen du clavier pour qu’on s’assure qu'il est utilisable par les lecteurs d'écran.

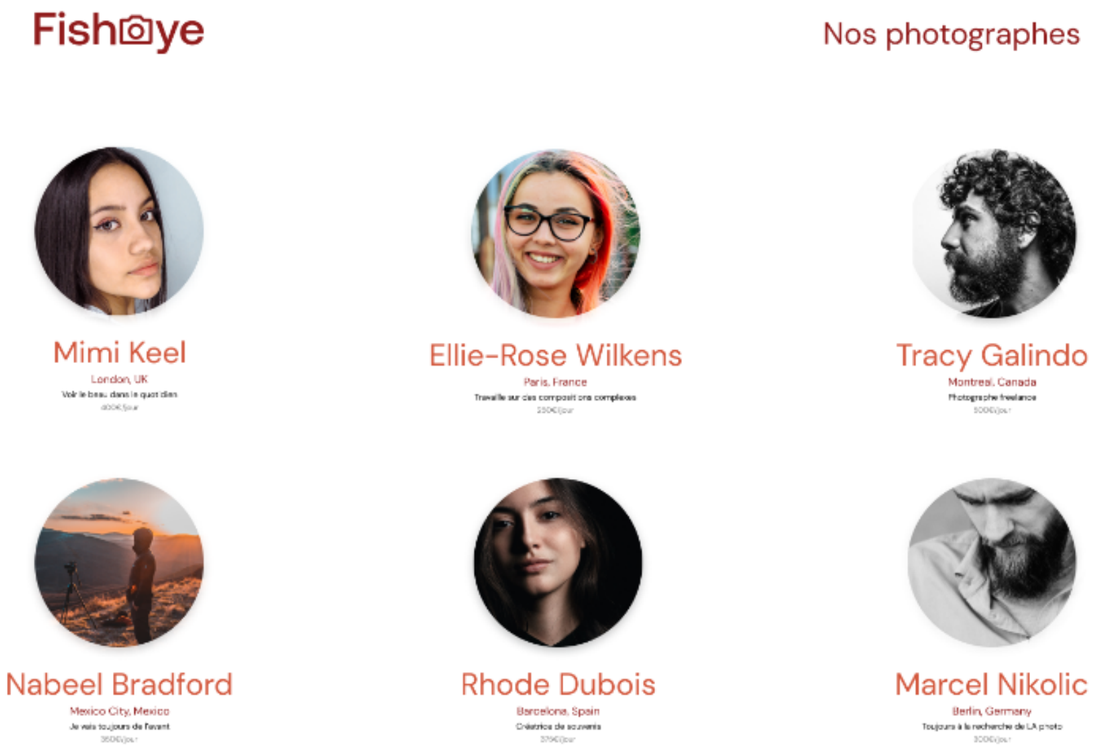
De plus, avant de soumettre ton travail, assure-toi qu'il respecte bien les contraintes techniques indiquées dans mes notes de réunion.

Bonne chance et montre-moi ce que tu sais faire !

Amanda

Cheffe de Projet @Techasite

Vous commencez par ouvrir les maquettes pour avoir une idée de ce qui doit être fait :



- Lorsque vous commencez à regarder les maquettes, vous recevez un message instantané de Zoé, une développeuse senior que vous admirez beaucoup. Elle vous a encadré et a effectué un grand nombre de vos révisions de code :

Zoé : Salut ! Amanda m'a dit qu'elle t'avait chargé de développer le premier prototype du projet FishEye, c'est bien ça ?

Moi : Oui, je viens de commencer !

Zoé : Tant mieux pour toi ! J'avais commencé ce projet, mais on m'a passé sur autre chose. Je pense que tu apprendras beaucoup au cours du processus. Je voulais juste te donner un petit conseil technique. Amanda a mentionné que FishEye a des vidéos et des photos pour le photographe. Pour ça il faudra que tu utilises un pattern Factory Method, ça sera idéal pour créer les media en distinguant les vidéos des photos.

Moi : OK, je vais utiliser le pattern Factory Method. Merci encore !

Photographer.html

Photographer.html

Ce code HTML représente une page web pour un site de photographie. Il comprend les éléments suivants :

1. La déclaration du type de document et la langue du document.
2. La section <head> qui contient des métadonnées, des liens vers des fichiers CSS, une icône pour le site, une police Google et le titre de la page.
3. La section <body> qui contient le contenu principal de la page.
4. L'en-tête de la page qui contient un logo avec un lien vers la page d'accueil.
5. La section principale qui contient des informations sur le photographe, un menu déroulant pour trier les photos, une galerie de photos, une lightbox pour afficher les photos en grand format, et des encarts pour les likes et les prix.
6. Un formulaire de contact pour contacter le photographe.
7. Des scripts JavaScript pour gérer différentes fonctionnalités de la page.
8. Un lien vers la bibliothèque Font Awesome pour utiliser des icônes.

En résumé, ce code est une page web qui présente une galerie de photos pour un photographe, avec la possibilité de trier les photos, d'afficher les photos en grand format et de contacter le photographe via un formulaire.

Photographer.html

```html
<!DOCTYPE html>
<html lang="fr">
  <head>      You, il y a 6 jours • commit
    <meta charset="utf-8">
    <link href="css/style.css" rel="stylesheet" type="text/css">
    <link rel="stylesheet" href="css/photographer.css">
    <link rel="icon" type="image/png" href="assets/favicon.png">
    <link href="https://fonts.googleapis.com/css?family=DM+Sans&display=swap" rel="stylesheet">
    <title>Fisheye - photographe </title>
  </head>
  <body>

    <header>
      <a href="index.html" class="logoHeader">
        <img src="assets/images/logo.png" class="logo" alt="Fisheye Home Page">
      </a>
    </header>

    <main id="main" aria-hidden="false">

      <!-- Informations Photographe -->
      <div class="photograph-header">
        <button class="infos-contact-button" onclick="displayModal()">Contactez-moi</button>
      </div>

      <!-- Menu select de tri des photos -->
      <div class="select-container">
        <label for="select">Trier par</label>
        <select name="tri" id="select">
          <option value="popularite" tabindex="0">Popularité</option>
          <option value="date" tabindex="0">Date</option>
          <option value="titre" tabindex="0">Titre</option>
        </select>
      </div>
```

```html
<!-- Gallerie Photos -->
<div class="photographer-galery"></div>

<!-- Lightbox -->
<div class="lightbox-container" role="dialog" aria-hidden="true" aria-label="li
  <ul class="lightbox" id="myDiv2">
    <!-- Gallerie d'images -->
  </ul>
  <button class="lightbox-close" aria-label="Fermer"></button>
  <a class="slide lightbox-prev" aria-label="Photo précédente">
    <i class="fa fa-chevron-left"></i>
  </a>
  <a class="slide lightbox-next" aria-label="Photo suivante">
    <i class="fa fa-chevron-right"></i>
  </a>
</div>
<div class="lightbox-bg"></div>

<div id="medias">
  <!-- Injecter en js la mosaique des images -->
</div>

<div id="likes-price">
  <!-- Injecter en js l'encart dédié aux likes et au prix-->
</div>
</main>
```

Photographer.html

Fishoye

Nos photographes

Mimi Keel
London, UK
Voir le beau dans le quotidien
400€/jour

Ellie-Rose Wilkens
Paris, France
Capturer des compositions complexes
250€/jour

Tracy Galindo
Montreal, Canada
Photographe freelance
500€/jour

Nabeel Bradford
Mexico City, Mexico
Toujours aller de l'avant
350€/jour

Rhode Dubois
Barcelona, Spain
Je crée des souvenirs
275€/jour

Marcel Nikolic
Berlin, Germany
Toujours à la recherche de LA photo
300€/jour

contactForm.js

contactForm.js

Ce code JavaScript gère l'affichage et la fermeture d'une modal de contact dans une page web. Voici un résumé des fonctionnalités principales :

1. La constante `modal` est utilisée pour stocker la référence de l'élément HTML correspondant à la modal de contact avec l'ID "contact-modal".
2. La fonction `displayModal()` est appelée pour afficher la modal de contact. Elle modifie la propriété CSS `display` de la modal pour la rendre visible ("block"). De plus, si les attributs `aria-hidden` de la modal et de l'élément principal (`main`) existent, ils sont modifiés pour indiquer l'état de visibilité approprié.
3. La fonction `closeModal()` est appelée pour fermer la modal de contact. Elle modifie la propriété CSS `display` de la modal pour la cacher ("none"). De plus, si les attributs `aria-hidden` de la modal et de l'élément principal (`main`) existent, ils sont modifiés pour indiquer l'état de visibilité approprié.
4. Un écouteur d'événement est ajouté à la fenêtre (`window.addEventListener`) pour détecter la pression de la touche "Escape". Si la modal a l'attribut `aria-hidden` avec la valeur "false" (c'est-à-dire qu'elle est visible) et si la touche pressée est "Escape", alors la fonction `closeModal()` est appelée pour fermer la modal.

En résumé, ce code permet d'afficher et de fermer une modal de contact en réponse à des événements tels que le clic sur un bouton ou la pression de la touche "Escape". Il utilise les attributs `aria-hidden` pour gérer l'accessibilité de la modal et pour gérer la visibilité de l'élément principal lorsque la modal est affichée.

contactForm.js

```javascript
const modal = document.getElementById("contact-modal");

// Afficher la modal de contact
function displayModal() {
    const modal = document.getElementById("contact-modal");
    modal.style.display = "block";
        if (modal.hasAttribute("aria-hidden") && main.hasAttribute("aria-hidden")) {
            modal.setAttribute("aria-hidden", "false");
            main.setAttribute("aria-hidden", "true");
        }
        const prenom = document.querySelector('#prenom');
        prenom.focus();
}

// Fermeture de la modal de contact
function closeModal() {
    const modal = document.getElementById("contact-modal");
    modal.style.display = "none";
        if (modal.hasAttribute("aria-hidden") && main.hasAttribute("aria-hidden")) {
            modal.setAttribute("aria-hidden", "true");
            main.setAttribute("aria-hidden", "false");
        }
}

// Fermeture avec la touche échap
window.addEventListener("keyup", (e) => {
    if (modal.getAttribute("aria-hidden") === "false" && e.key === "Escape") {
        closeModal();
    }
});
```

Photographer.js

1. Récupération des identifiants des photographes dans l'URL à l'aide de window.location.search et URLSearchParams.
2. Initialisation de l'index lightboxIdx pour la fonctionnalité de la "lightbox".
3. Récupération des données des différents photographes à partir d'un fichier JSON à l'aide de la fonction asynchrone getPhotographer() qui utilise fetch.
4. Récupération des données MEDIA des différents photographes à partir du même fichier JSON à l'aide de la fonction asynchrone getMedia().
5. La fonction displayDataPhotographer(photographer) est appelée pour afficher les données liées au profil du photographe.
6. La fonction displayDataGalery(media) est appelée pour afficher les photos dans la galerie dédiée.
7. La fonction displayDataLightbox(medias) est appelée pour afficher les données dans la "lightbox".
8. La fonction sortMedia(media, triValue) est utilisée pour trier les médias en fonction de la valeur sélectionnée dans un élément select.
9. L'événement change est ajouté à l'élément select pour déclencher le tri et afficher les médias triés dans la galerie et la "lightbox".
10. La fonction setLightboxEvents(mediaLength) est appelée pour gérer les événements de la "lightbox" tels que la fermeture, la navigation précédente/suivante.
11. La fonction setGalleryEvent() est appelée pour gérer l'affichage de la "lightbox" en fonction de l'index des photos de la galerie.
12. La fonction init() est appelée pour récupérer les informations du photographe, afficher les données et gérer les événements.

```
// Récupére les identifiants des photographes dans l'URL
const queryString = window.location.search;
const urlParams = new URLSearchParams(queryString);
const photographerId = parseInt(urlParams.get("id"));

// L'initialisation de l'index photo "Lightbox"
let lightboxIdx = 0;

// Récupére les datas des différents photographes avec un fetch
async function getPhotographer() {
    const photographers = await fetch("./data/photographers.json")
        .then((res) => res.json())
        .then(data => data.photographers)
    return {
        photographer: photographers.find(p => p.id === photographerId)
    };
}

// Récupére les datas MEDIA des différents photographes avec un fetch
async function getMedia() {
    await fetch("./data/photographers.json")
        .then((res) => res.json())
        .then((data) => (media = data.media));
    return {
        media: media.filter(m => m.photographerId === photographerId)
    };
}
      You, il y a 6 jours • commit
// Page photographe : Afficher les datas liées au profil des protographes
async function displayDataPhotographer(photographer) {
    photographerInfosHeader(photographer);
    photographerInfosContact(photographer)
}
```

```
/ Page photographe : Afficher les datas liées au profil des protographes
sync function displayDataPhotographer(photographer) {
   photographerInfosHeader(photographer);
   photographerInfosContact(photographer)


/ Affiche les photos dans la gallerie dédiée
sync function displayDataGalery(media) {
   const photographerGalery = document.querySelector('.photographer-galery');
   photographerGalery.innerHTML = '';
   media.forEach((itemMedia, idx) => {
       const photographerGaleryModel = photographerMediaFactory(itemMedia);
       const userGaleryCardDOM = photographerGaleryModel.CreateGaleryDom(idx);
       photographerGalery.appendChild(userGaleryCardDOM);
   })


/ Affichage de la Lightbox
unction displayDataLightbox(medias) {
   const lightbox = document.querySelector('.lightbox');
   lightbox.innerHTML = '';
   medias.forEach(itemMedia => {
       const photographerLightboxModel = photographerMediaFactory(itemMedia);
       const userLightboxDOM = photographerLightboxModel.createLightboxDOM();
       lightbox.appendChild(userLightboxDOM);
   })
```

index.js

La fonction asynchrone getPhotographers() est définie pour récupérer les données des différents photographes à partir d'un fichier JSON à l'aide de fetch. Les données récupérées sont ensuite stockées dans une variable photographers.

La fonction displayData(photographers) est définie pour afficher les cartes des photographes dans la section dédiée. Elle parcourt chaque photographe et utilise une fonction photographerFactory(photographer) pour créer et ajouter la carte du photographe à la section des photographes.

La fonction init() est définie pour compiler les fonctions de récupération et d'affichage des données. Elle appelle la fonction getPhotographers() pour obtenir les données des photographes, puis appelle la fonction displayData(photographers) pour les afficher.

La fonction init() est appelée pour démarrer le processus de récupération et d'affichage des données des photographes.

Ce code semble être destiné à une page web qui affiche une liste de photographes à partir d'un fichier JSON. Il utilise la fonction fetch pour récupérer les données, puis génère et affiche les cartes des photographes en utilisant une fonction de création de carte spécifique (photographerFactory).

```javascript
// Récupérer les datas des différents photographes avec un fetch
async function getPhotographers() {
    await fetch("./data/photographers.json")
    .then((res) => res.json())
    .then((data) => (photographers = data.photographers));
  return {
    photographers: [...photographers]
  };
  }

  // Fonction pour afficher les Cards photographes
  async function displayData(photographers) {
    const photographersSection = document.querySelector(".photographer_section");

    photographers.forEach((photographer) => {
        const photographerModel = photographerFactory(photographer);
        const userCardDOM = photographerModel.getUserCardDOM();
        photographersSection.appendChild(userCardDOM);
    });
  };

  // Fonction pour la compilation des fonctions de récupération et d'affichage des datas
  async function init() {
    // Récupère toutes les datas des photographes
    const { photographers } = await getPhotographers();
    displayData(photographers);
  };
        You, il y a 6 jours • README
  init();
```

Les factories

media.js

La factory mediaFactory est une fonction qui crée et retourne un objet représentant un média (une image ou une vidéo). Cette factory utilise les données fournies pour déterminer le type de média (image ou vidéo) et générer l'élément HTML correspondant.

Résumé de la factory mediaFactory :

- La fonction mediaFactory prend un objet data en tant que paramètre.
- Elle détermine le type de média en vérifiant si la propriété video de l'objet data existe.
- La fonction interne createElement crée l'élément HTML correspondant au type de média.
- Si le type est une "image", la fonction génère une balise img avec les attributs appropriés (source de l'image, attribut alt, etc.).
- Si le type est une "vidéo", la fonction génère une balise video avec les attributs appropriés (source de la vidéo, attribut alt, etc.).
- La fonction retourne un objet avec deux propriétés : type (le type de média) et createElement (une fonction qui génère l'élément HTML du média).
- L'objet retourné par la factory peut être utilisé pour créer et manipuler des médias dans le code.

```
function mediaFactory(data) {
    let type = data.video ? "video" : "image";

    function createElement() {
      let element;
      if (type == "image") {
        element = `<img src=assets/media/${data.photographerId}/${data.image} alt="${data.title}" data-id=${data.id}></img>`;
      } else if (type == "video") {
        element = `<video src=assets/media/${data.photographerId}/${data.video}#t=0.1 alt="${data.title}" data-id=${data.id} preload="metadata"></vi
        //#t=0.1 et preload="metadata" pour que l'image de la video s'affiche sur safari
      }

      const li = `
                <li class="media" data-date=${data.date} data-likes=${data.likes} data-title=${data.title}>
                  <a href="#" class="media__link" onclick="openLightbox('${data.id}')">
                    ${element}
                  </a>
                  <div class="media__info">
                    <p>${data.title}</p>
                    <button class="like" onclick="likeEvent(this)" >
                      <p>${data.likes}</p>
                      <img src="assets/icons/heart-vide.svg" id="coeurVide" alt="coeur-vide"/>
                      <img src="assets/icons/heart.svg" id="coeur" alt="coeur"/>
                    </button>
                  </div>
                </li>
              `;
      return li;
    }
    return { type, createElement };
}      You, il y a 6 jours • commit
```

La factory mediaFactory permet donc de créer des objets représentant des médias avec une structure cohérente et fournit une méthode pour générer l'élément HTML du média à partir des données fournies.

mediaPhotographer.js

1. La fonction photographerInfosHeader est utilisée pour créer l'en-tête des informations du photographe. Elle crée des éléments HTML tels que le nom, la ville, le pays et le slogan du photographe, et les ajoute à l'en-tête correspondant dans le document.
2. La fonction photographerInfosContact est utilisée pour mettre à jour la modal de contact avec les informations du photographe, en particulier le nom du photographe.
3. La classe MediaFactory est une factory pour générer des éléments HTML représentant des médias (images ou vidéos). La méthode render crée et retourne l'élément HTML approprié en fonction du type de média (image ou vidéo).
4. La fonction photographerMediaFactory est une factory qui crée des éléments HTML pour représenter les médias d'un photographe. Elle génère des éléments tels que des vignettes d'images ou de vidéos, des titres, des boutons de likes, etc. La factory utilise la classe MediaFactory pour générer les éléments correspondants aux médias.
5. La fonction getLikesPrice est utilisée pour afficher le nombre total de likes pour tous les médias d'un photographe, ainsi que le prix journalier du photographe. Elle calcule également le total des likes à partir des données des médias.
6. La fonction displayLightbox est utilisée pour afficher la modal lightbox en définissant le style CSS approprié.

```javascript
// Fonction de création des Cards photographes contenant leurs informations personnelles
function photographerFactory(data) {
    const { name, portrait, city, country, tagline, price, id } = data;
    const picture = `assets/photographers/${portrait}`;

    // Création du DOM des Cards photographe --- AJOUTER PP POUR LA CONSTITUTION DE DOM
    function getUserCardDOM() {
        const article = document.createElement( "article" );
        const img = document.createElement( "img" );
        const a = document.createElement( "a" );
        a.setAttribute("href", `photographer.html?id=${id}`);
        img.setAttribute("src", picture);
        img.setAttribute("alt", name);
        const h2 = document.createElement( "h2" );
        h2.textContent = name;
        const location = document.createElement( "p" );
        location.textContent = `${city}, ${country}`;
        const tagLine = document.createElement( "span" );
        tagLine.classList.add("tagLine");
        tagLine.textContent = tagline;
        const prices = document.createElement( "span" );
        prices.textContent = `${price}€/jour`;
        prices.classList.add("prices");

        a.appendChild(img);      You, il y a 6 jours • commit
        a.appendChild(h2);
        article.appendChild(a);
        article.appendChild(location);
        article.appendChild(tagLine);
        article.appendChild(prices);

        return (article);
    }

    return { name, picture, getUserCardDOM };
}
```

Cette factory est utilisée pour créer les cards des photographes en utilisant leurs données respectives.

Photographer.js

1. La factory prend en entrée un objet data contenant les informations du photographe, telles que le nom, le portrait, la ville, le pays, le slogan, le prix et l'identifiant.
2. La méthode getUserCardDOM est utilisée pour créer et retourner l'élément HTML représentant la card du photographe. Elle crée des éléments tels qu'un article, une image, un lien, un titre, une localisation, un slogan et un prix. Les différentes informations du photographe sont extraites de l'objet data et ajoutées aux éléments HTML correspondants.
3. L'élément image contient le chemin vers l'image du portrait du photographe, et le lien est défini pour rediriger vers la page du photographe correspondant à son identifiant.
4. Finalement, les différents éléments sont ajoutés à l'article, qui est ensuite retourné.
5. La factory photographerFactory retourne un objet contenant le nom du photographe, le chemin de l'image du portrait et la méthode getUserCardDOM pour créer la card du photographe.

Fin de la Soutenance Projet OpenClassrooms

Fishoye

Créez un site accessible pour une plateforme de photographes

Présenter par Ghani